TDEF-YOLO: A powerful surface defect detector

Jian Li, Hua Li*, Yixiong Feng, Songrong Qian, XiangKun Hu

[a] *Guizhou University State Key Laboratory of Public Big Data, , GuiYang, China*

Abstract

Surface defects significantly impact product quality and reliability, necessitating robust detection solutions. Conventional vision-based methods suffer from environmental sensitivity and limited generalization, while deep learning approaches face challenges in few-shot adaptability, real-time performance, and multi-scale defect handling. This paper introduces TDEF-YOLO, an optimized YOLOv5 architecture integrating three technical innovations: 1) a lightweight EfficientViT backbone with cascaded group attention, reducing parameters to 3.95M and GFLOPs to 7.6G; 2) a deformable convolution-multi-scale attention neck module enhancing adaptive feature fusion; 3) a Focal-CIoU loss dynamically balancing hard/easy sample contributions.On the NEU-DET dataset, TDEF-YOLO achieves 82.9% mAP@0.5 with 104 FPS, representing a 6.5% accuracy improvement over YOLOv5 while reducing parameters by 45% and GFLOPs by 54%. Ablation analysis validates component effectiveness, and results on the PKU-Market-PCB dataset (95.6% mAP) confirm generalization. This approach provides a balanced solution for automated surface inspection through architectural innovations, achieving superior efficiency-accuracy tradeoffs.

Keywords: Surface defect detection, YOLOv5, EfficientVIT, Deformable Convolution, Efficient Multi-Scale Attention

1. Introduction

In all kinds of industrial products, whether the surface defects of products can reflect the performance of their products to a large extent, surface defect detection is an important means to eliminate defective products, and also an important basis for measuring technological progress. Due to the influence of processing environment, equipment and environment, the surface of products often contains various defects, such as stains and damage on textiles, cracks, scratches and inclusions on metal products. There are many kinds of defects, and the differences in product size, shape, texture and surface reflectivity are huge, which makes it particularly difficult to detect surface defects in the real environment. The exploration of surface defect detection dates back to the 1950s. At that time, it was mainly through online observation of human eyes and offline sampling detection, with low efficiency. With the development of automation technology, the task of defect detection is gradually transferred to the machine, which greatly improves the work efficiency(1; 2; 3) . Studying complex defect detection algorithms is crucial to improve production efficiency and quality.

With the development of machine learning, defect detection using machine learning has been gradually studied and applied, which is mainly divided into two stages : feature extraction and defect recognition. For example, Win et al.(4) proposed an Otsu 's method based on contrast adjustment and a median Otsu's method based on contrast adjustment to construct a titanium coating surface defect detection system. A defect detection algorithm based on Fuzzy C-Means Clustering (IFCM)(5). Unsupervised defect detection algorithm based on machine learning method(6) and defect detection algorithm based on Entity Sparse Tracking (ESP) method(7). However, these methods have limitations, mainly including the following aspects. (1) It is necessary to extract features and design classifiers manually, which is cumbersome to operate; (2) Harsh environmental requirements, relying on good light source and imaging conditions; (3) It is powerless to deal with complex defects. In contrast, deep learning methods are more suitable for defect recognition tasks under complex conditions because they do not require hand-designed features and classifiers, and are more widely used in surface defect detection.

Currently, significant progress has been made in the field of computer vision (CV) due to the powerful automatic feature extraction capabilities of convolutional neural networks (CNN). This success covers a variety of applications, including key point detection, target tracking, intelligent driving, etc. Compared with machine learning methods, deep learning methods show obvious advantages in feature extraction and adaptability to different environments. In 2014, D. Soukup et al.(8) applied convolutional neural networks to defect detection for the first time, using fully supervised trained CNNs for experiments, and proved that classical CNNs are significantly superior to machine learning-based methods. Subsequently, deep convolutional neural networks (DCNN) are increasingly integrated into object detection networks, including Faster-

Email address: `Corresponding author,lihua.41@163.com` (Hua Li*)

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5205180

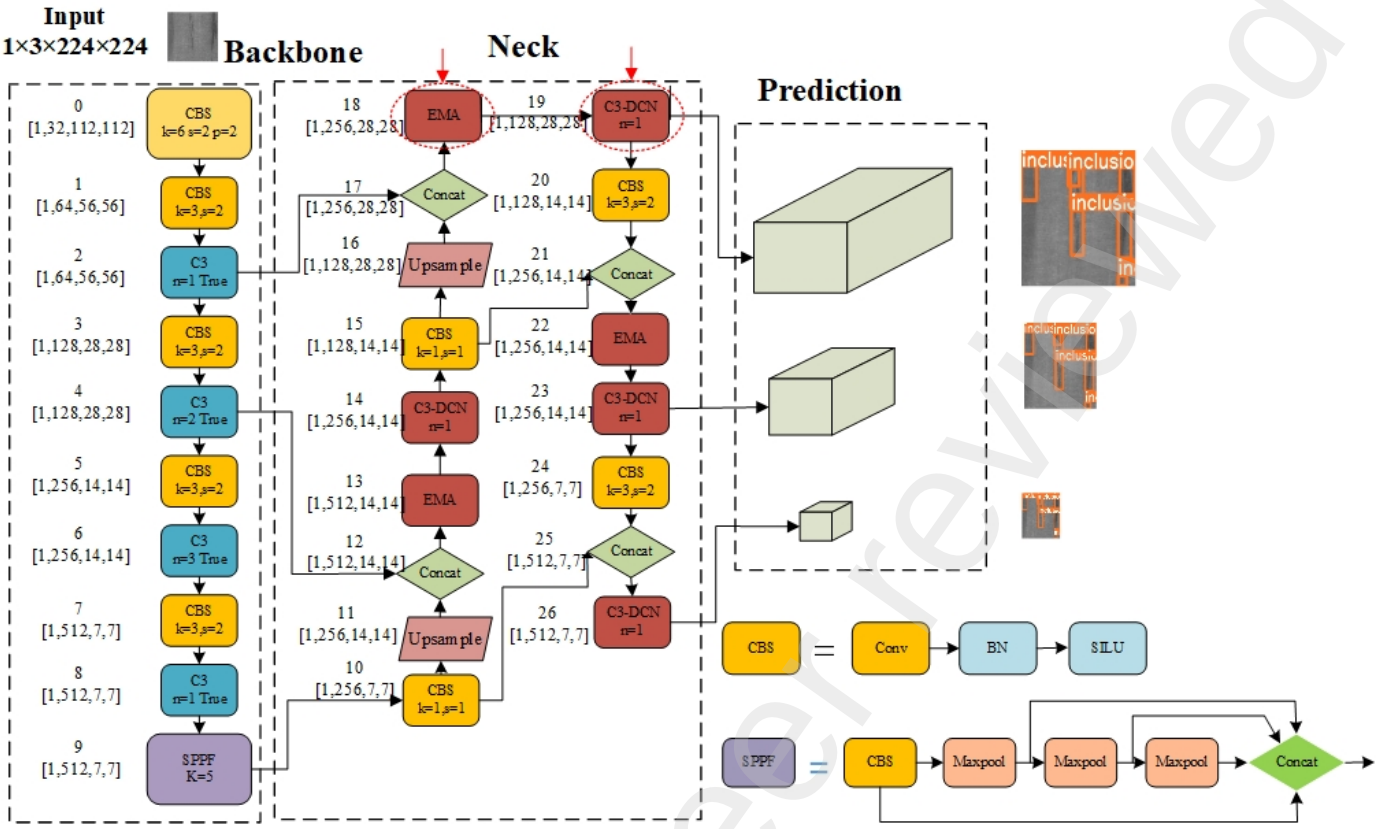


Figure 1: TDEF-YOLO network model. The EMA, and C3-DCN circled in the ellipse in the figure are newly added modules.

RCNN(9), You Only Look Once (YOLO) series(10; 11; 12) and SSD(13). Su et al.(14) constructed a Faster R-CNN network with attention mechanism, and achieved an average accuracy of 87.38% in solar panel defect detection. Yin et al.(15) used an advanced convolutional neural network based object detector YOLOv3 network, and achieved an average accuracy of 85.37% in the water pipe defect detection task. Considering the processing speed and accuracy, it is obvious that this algorithm is very effective in the field of object detection. Jiang et al.(16) optimized the SSD target detection algorithm by means of deep separable convolution, reverse residual network and linear bottleneck structure. The network achieved 64.81% accuracy in the complete detection of pavement defects.

In recent years, research on visual inspection of metal products has gradually emerged. Based on traditional machine learning methods. Xiao et al.(17) introduced an evolutionary classifier with Bayesian kernel (BYEC) and achieved a classification accuracy of 96.3% on NEU-CLS. In (18), The classifier adopts a deep convolutional neural network classification method and utilizes pre-trained model knowledge to achieve a classification accuracy of 99.27% on NEU-CLS. However, the above method only provides a solution to the problem of defect classification, and does not detect the defect area in a deeper level, which is not enough to intuitively find the defect location. To more effectively achieve strip surface defect detection, Zhao et al.(19) proposed a defect detection model called RDD-YOLO. Although the model achieved an accuracy of 81.1%, the improvement in accuracy was not particularly significant. Therefore, in (20), a Defect Detection Network (DDN) using ResNet50 as the backbone network is proposed, achieving a maximum accuracy of 82.3%, but its detection speed fails meet the real-time requirements.

YOLOv5, an upgraded version of the YOLO series, offers higher detection accuracy and faster speed compared to its predecessors, effectively meeting the real - time requirements of actual industrial detection. It is currently operated and maintained by the official GitHub account of Ultralytics. Although YOLOv5 is an advanced detection framework, it has several limitations. Its lack of a prominent lightweight design makes it challenging to meet real - time requirements. The defined loss function fails to effectively enhance the training capacity. Additionally, the backbone feature extraction ability is sub - optimal, and the head feature fusion ability is weak, leading to low accuracy in simple datasets. To address these limitations, this study presents TDEF-YOLO, an optimized YOLOv5 architecture integrating three technical innovations:

1 To reduce model parameters, enhance detection speed, and control accuracy degradation, a lightweight backbone architecture is adopted to replace the original structure through network depth streamlining and convolutional operation optimiza-

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5205180

Table 1: CSPTaknet parameter

Network Layer	Input Stem	Stage1	Stage2	Stage3	Stage4
Number of Replications	1	1	1	1	1
Characteristic Parameter	[32,112,112]	[64,56,56]	[128,28,28]	[256,14,14]	[512,7,7]

tion, thereby minimizing redundant computations while maintaining feature extraction capabilities and enabling real-time high-accuracy detection for industrial applications.

2 To address fixed receptive field limitations of traditional rectangular convolution kernels, deformable convolutional networks (DCN) are embedded into the C3 module of the model neck for adaptive sampling of irregular defect morphologies. This is complemented by attention layers with dynamic weight allocation to prioritize informative regions over irrelevant interference. These modifications strengthen multi-scale feature fusion and extraction capabilities across different spatial dimensions, improving detection accuracy for complex industrial defects.

3 To enhance the regression speed and accuracy of the CIoU loss function, a penalty factor incorporating the Focal concept is introduced to dynamically balance sample contributions, enabling the model to prioritize common sample feature learning while reducing abnormal sample interference and optimizing the regression process, thereby improving detection box positioning accuracy and convergence speed for enhanced detection stability and accuracy in complex scenarios.

2. Proposed Method

The comprehensive network architecture of TDEF-YOLO is shown in Figure 1. Taking a JPG format image with an input size of 224×224 as an example, the image passes through the backbone network to generate feature maps of different sizes and dimensions. Subsequently, feature fusion is performed when entering the neck. Finally, features at three different scales (large, medium, and small) are forwarded to the inspection head for defect identification. This is a multi-scale, real-time and efficient defect detection algorithm..

2.1. Strategies for the Backbone

The backbone network has developed rapidly, and there are many networks to choose from, including VGGNet(21), ResNet(22), MobileNet(23), EfficienNet(24), etc. While YOLOv5 employs CSPDaknet as its backbone network, integrating advantages of previous architectures, it still faces challenges such as large parameter counts and high computational complexity. To address these limitations, EfficientVIT is adopted as the backbone network for YOLOv5. Table 1 presents the parameters configuring output feature maps for each layer of the backbone network.

EfficientViT, a novel Transformer architecture tailored for visual tasks, introduces three key innovations: Extensive Lightweight Convolutional Structures: The network incorporates Depthwise Sparsely Quantized Convolution (DSConv), Depthwise Convolution (DWConv), and Mobile Inverted Bottleneck Convolution (MBConv) to replace traditional convolutions. This design significantly reduces parameter counts while maintaining feature representation capabilities. Sandwich Layout: By inserting multi-head self-attention layers between Feed-Forward Network (FFN) blocks, this layout minimizes temporal overhead. Increasing FFN layer depth enhances channel communication and memory efficiency, enabling the model to operate effectively under constrained computational resources. Cascaded Group Attention Module: Features are partitioned into distinct segments for individual attention heads, avoiding redundant computations from identical input features. This approach not only reduces computational costs but also increases network depth and attention diversity, facilitating better extraction of multi-scale visual features.

On the right side of Figure 2, the operation logic of the sandwich layout and cascaded attention module is presented. After input data is processed by the Linear layer, it is split into Q, K, and V groups, then enters the three-layer parallel sandwich layout structure. The first path directly applies ReLU Linear Attention, leveraging an efficient attention mechanism to process features, preserving basic feature information and rapidly completing preliminary feature screening. The second path first goes through 3×3 DWConv (Depthwise Convolution). With its lightweight computation property, it collects global information and expands feature perception scope. Then, 1×1GConv (1×1 common convolution) encodes features to adjust channel dimensions, followed by ReLU Linear Attention optimizing feature weights to highlight key information. The third path shares a similar process with the second one but adopts 5×5 DWConv. The larger convolution kernel endows it with a wider receptive field, excelling in extracting large-target features and capturing macroscopic structure information in images. Afterward, it also undergoes 1×1 common convolution encoding, then completes feature weighting via ReLU Linear Attention. Once the three-path results are aggregated, a fully connected Linear layer integrates multi-scale feature information for final output. By setting differentiated convolution kernels and combining with the attention mechanism, this mod-

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5205180

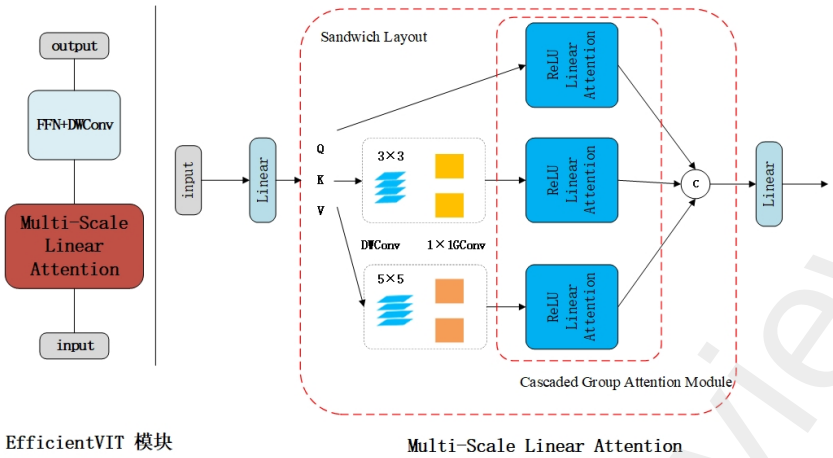


Figure 2: Multi-Scale Linear Attention and EfficientVIT Module

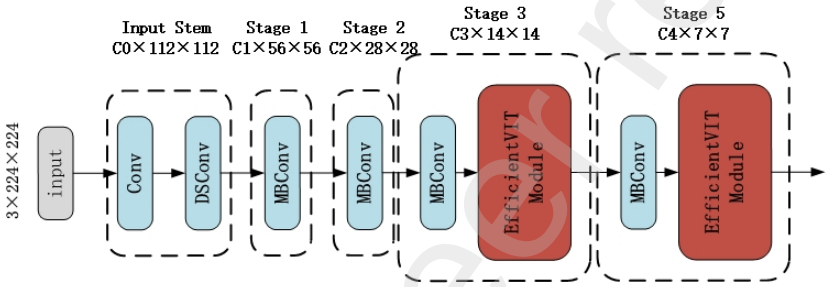


Figure 3: EfficientVIT

ule—termed the multi-scale connection attention mechanism—achieves multi-scale feature extraction and fusion, significantly improving the model's detection adaptability and accuracy for targets of varying sizes.

On the left side, the EfficientVIT Module—a basic module built based on EfficientVIT—is shown. Its operation logic is as follows: Input data first enters the multi-scale connection attention mechanism layer. This layer realizes multi-scale feature fusion through differentiated convolution kernel settings and attention calculation, allowing the model to capture both subtle local features and global semantic information. After processing by the multi-scale connection attention mechanism, data flows into the Feed-Forward Neural Network (FFN). FFN conducts complex semantic transformations on features via linear transformations and nonlinear activation functions, exploring deep feature correlations and enhancing feature expression richness. Subsequently, data enters the DWConv (Depthwise Convolution) layer. DWConv uses lightweight convolution to specifically extract local spatial features, preserving image spatial structure information while reducing computation. Finally, after three layers of processing—multi-scale connection attention mechanism-driven feature fusion, FFN-based semantic transformation, and DWConv-based local feature extraction—the module outputs results integrating multi-scale semantic information, complex feature relationships, and local spatial details, providing a high-quality feature foundation for subsequent tasks like object detection and image classification.

Figure 3 demonstrates the architecture of the EfficientVIT network. Taking an input image with a size of 3×224×224 as an example, the image first enters the Input Stem module, undergoing Conv (common convolution) and DSConv (depthwise separable convolution) successively to complete initial feature extraction. Subsequently, the image proceeds through four stages of the network (Stage 1 to Stage 4). Stage 1 consists of MBConv modules stacked repeatedly (×L times), gradually compressing the feature map size and enhancing feature abstraction. Stage 2 also employs MBConv module stacking (×L times) to further adjust feature dimensions. Stage 3, based on the stacking of MBConv and EfficientVIT Module (×L times), strengthens the multi-scale feature fusion capability. Stage 4 integrates MBConv and EfficientVIT Module stacking (×L times) to complete final feature processing. After the full network computation, feature maps of specific sizes are ultimately output. Notably, the input and output sizes of EffcientVIT perfectly match those of CSPDarknet53, the original backbone network of YOLOv5. This design significantly simplifies the model replacement process. When replacing CSPDarknet53 with EffcientVIT, no additional parameter modifications are required, enabling seamless integration. It not only maintains the compatibility of the YOLOv5 framework but also provides a convenient up-

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5205180

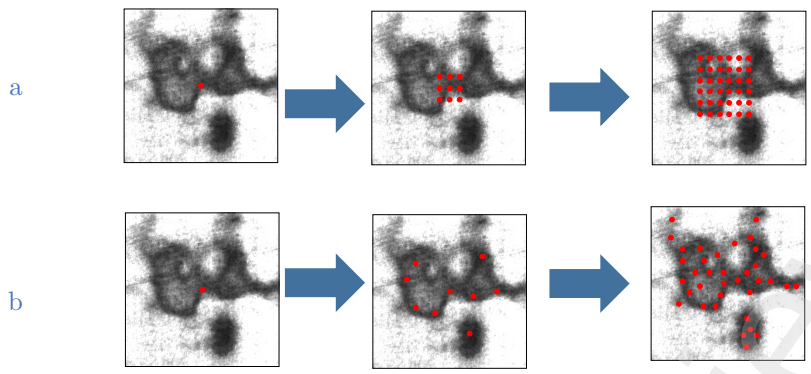


Figure 4: a represents the traditional convolution, which is a fixed rectangular structure ; b represents deformable convolution, and its kernel is not limited to fixed sampling points.

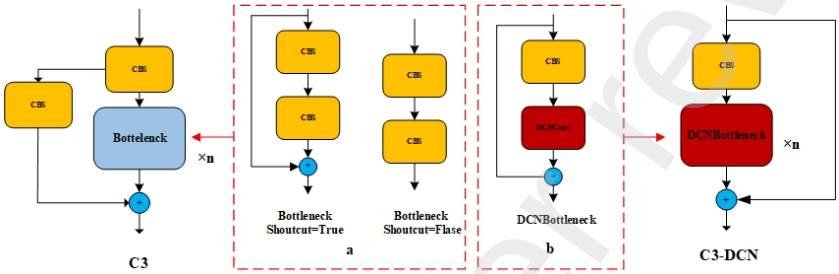


Figure 5: C3 and C3-DCN

grade path for model performance optimization, avoiding complex adaptation issues caused by parameter adjustments.

2.2. Strategies for the Neck

Conventional convolutional neural networks (CNNs) rely on static rectangular kernels with fixed sampling grids, inherently limiting their ability to model geometric variations and complex object morphologies. To address these limitations, Dai et al. (25) pioneered deformable convolution—a paradigm shift introducing learnable offset parameters that dynamically adjust sampling positions within the receptive field (Fig. 4). This innovation significantly enhances spatial adaptation, particularly when processing high-dimensional feature maps containing intricate defect patterns, by enabling context-aware deformation of convolutional operators rather than rigid geometric constraints.

In traditional convolution kernels, the convolution representation of each pixel in the input image at position p_0 is as follows:

$$y(p_0) = \sum_{p_0=R} w(p_n) \times x \times (y(p_0) + y(p_n)) \tag{1}$$

The offset variable Δp_k and penalty term are Δm_k introduced into the deformable convolution, which is obtained by convolution learning. The deformable convolution operation for each pixel in the input image is mathematically formulated as:

$$y(p_0) = \sum_{p_0=R} w(p_n) \times x \times (y(p_0) + y(p_n) + \Delta p_k)) \times \Delta m_k \tag{2}$$

As shown in Fig. 5, the deformable convolution replaces the conventional convolution in the C3 bottleneck to form the DCN bottleneck. The C3-DCN module composed of DCN bottlenecks substitutes the original C3 module, where the discrete convolutional kernel is more conducive to effective feature extraction, thereby enhancing the neck's feature extraction capability.

The **Efficient Multi-Scale Attention (EMA)** module (Figure 6) consists of three core components, as described below. *Stage 1: Feature Grouping* partitions an input feature map $\mathbf{X} \in \mathbb{R}^{C\times H\times W}$ into G sub-features ($G \ll C$) along the channel axis, yielding $\mathbf{X} = [\mathbf{X}_0, \mathbf{X}_1, \ldots, \mathbf{X}_{G-1}]$ with each $\mathbf{X}_i \in \mathbb{R}^{(C/G)\times H\times W}$. *Stage 2: Parallel Subnetworks* process these sub-features through two branches: (1) The 1×1 branch encodes spatial dependencies via height/width decomposition, concatenates vectors, and applies Sigmoid activation to generate channel attention weights $\mathbf{W}_c = \sigma(\mathbf{f}_{1\times1}(\mathbf{X}_i))$, followed by multiplicative fusion $\mathbf{X}_{fused} = \mathbf{W}_c \odot \mathbf{X}_i$; (2) The 3×3 branch employs dilated convolution to model local inter-channel interactions and extract multi-scale features $\mathbf{F}_{spatial} = \mathbf{f}_{3\times3}(\mathbf{X}_i)$). *Stage 3: Cross-space Learning* integrates outputs from both branches: First, 2D global average pooling $Z_c = \frac{1}{H\times W}\sum_{j=1}^{H}\sum_{i=1}^{W} x_c(i,j)$ and softmax normalization are applied to the 1×1 branch output. Matrix dot products between this result and the raw 3×3 output generate dual spatial attention maps, which are summed, weighted by Sigmoid, and multiplied with the original input to produce refined features $\mathbf{Y} \in \mathbb{R}^{C\times H\times W}$. This tri-stage design ensures multi-scale feature enhance-

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5205180

ment while preserving computational efficiency.

Efficient Multiscale Attention (EMA) is a generalized technique that transforms channel dimensions into batch dimensions, outperforming the inherently dimensionality-reducing form of general convolution. Beyond enabling local cross - channel interactions within each parallel sub - network while maintaining the channel dimension, a cross - space (across different dimensions) learning strategy is employed to fuse the output feature maps from the two parallel sub - networks. This approach not only produces better results, but is also more parametrically efficient. Adding this module to the neck facilitates the interaction of information and reduces the loss of information, which improves the model's ability to adapt to complex scenes and increases the accuracy of detection.

2.3. Strategies for the Loss Function Optimization

The YOLOv5 loss function integrates three components: confidence loss L_{obj}, classification loss L_{cls}, and localization loss L_{loc}. While L_{obj} and L_{cls} employ cross-entropy to quantify prediction reliability and categorical accuracy, L_{loc} adopts the Complete Intersection over Union (CIoU) metric. CIoU extends conventional overlap measurement by jointly optimizing three geometric criteria: bounding box coverage, centroid proximity, and aspect ratio consistency. This multi-constraint design effectively mitigates gradient instability during training, enhancing regression robustness without manual feature engineering. The calculation formula as follows:

$$L_{CIoU} = 1 - IoU + \frac{\rho(b, b^{gt})}{c^2} + \alpha v \tag{3}$$

$$\alpha = \frac{v}{(1 - IoU) + v} \tag{4}$$

$$v = \frac{4}{\pi^2}(\arctan \frac{w^{gt}}{h^{gt}} - \arctan \frac{w}{h}) \tag{5}$$

where $IoU \in [0, 1]$ quantifies bounding box overlap between ground-truth (b^{gt}) and predicted (b) instances. $\alpha \geq 0$ denotes the balancing factor that weights geometric consistency penalties. v measures aspect ratio discrepancy between predicted (w, h) and ground-truth (w^{gt}, h^{gt}) dimensions. Spatial coordinates: $b = (x, y)$ and $b^{gt} = (x^{gt}, y^{gt})$ represent predicted and ground-truth centroids respectively.

The drawback of $CIoU$ is that it fails to balance positive and negative samples. During the training process, easily distinguishable samples and indistinguishable samples (outlier samples) are often treated equally. Indistinguishable samples can significantly impede the training performance. Although easily distinguishable samples consume computational resources, they cannot effectively improve the model's accuracy. Meanwhile, indistinguishable samples always drag down the model's training ability and reduce its accuracy. Focal Loss (26) was initially proposed for classification tasks. It alleviates the problem of sample imbalance by weighting the cross - entropy loss. In object detection tasks, there is an issue of imbalanced training samples, which is more prominent in one - stage detection. Owing to the sparse distribution of target objects in images, the quantity of high-quality instances with minimal regression errors and low-quality examples (outlier instances) with substantial errors is significantly lower than that of normal instances. Outlier examples may induce either negligible or excessive gradients, which impede the training process. Therefore, prioritizing gradient contributions from normal-quality instances during network training is essential to enhance model accuracy. To this end, the Focal concept is incorporated into $CIoU$. A weight term IoU^{γ} is assigned during the calculation, which is called $L_{Focal-CIoU}$. The calculation formula is as follows:

$$L_{Focal-CIoU} = IoU^{\gamma} \times L_{CIoU} \tag{6}$$

Where, $\gamma \in (0, 1)$ serves as a tunable focusing parameter. As shown in Figure 3-7, when x falls within the interval $(0.4, 0.8)$, the corresponding value increases significantly. This coefficient suppresses the influence of easy-to-classify samples and low-quality samples on the loss function while enhancing contributions from normal IoU samples, thereby improving training efficiency.

3. Experiment

The framework was rigorously evaluated on the *NEU-DET* benchmark for industrial surface defect analysis. All experiments were conducted on an *Ubuntu 18.04* workstation with 12-core *Intel Xeon Platinum 8255C* processors (2.50GHz) and *NVIDIA RTX 2080 Ti* GPUs (11GB VRAM). The computational pipeline leveraged *PyTorch 1.9.0* accelerated by *CUDA 11.1*, optimizing tensor operations through parallelized CUDA kernels for enhanced throughput.

3.1. Experimental details and datasets

The experiments utilize the NEU-DET benchmark dataset from Northeastern University, comprising 1,800 grayscale images (200×200 pixels) of steel strip surface defects. Six defect categories are equally represented, with 300 samples per class: crazing (Cr), inclusions (In), rolled-in scale (RS), plaque (Pa), pitting surface (PS), and scratches (Sc). Each image is annotated with defect category and spatial coordinates in XML format, which we convert to VOC-compatible annotations via custom scripts. To ensure statistical validity, the dataset undergoes an 8:1:1 stratified split into training, validation, and test partitions, maintaining proportional class distributions across subsets. Key configuration parameters include:

(1) The JPG format surface defect image with a size of 224*224 is input, and the YOLOv5s weight (14.1MB) trained on COCO2017 is used to transfer the backbone network weight.

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5205180

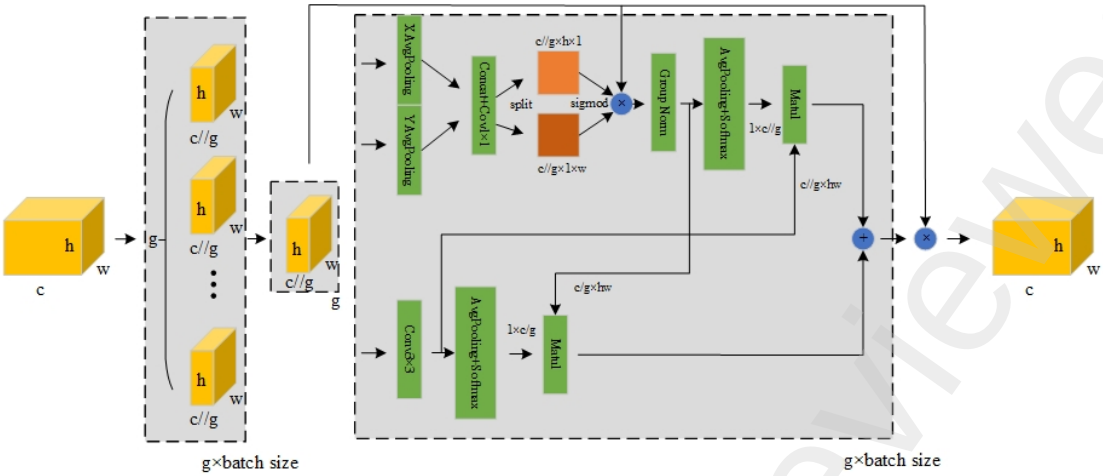


Figure 6: EMA. g represents the group size, AvgPool represents AdaptiveAvgPool2d, X represents the horizontal direction, and Y represents the vertical direction

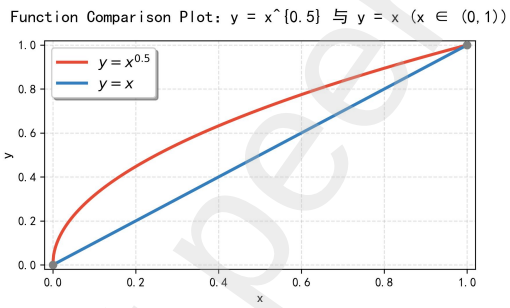
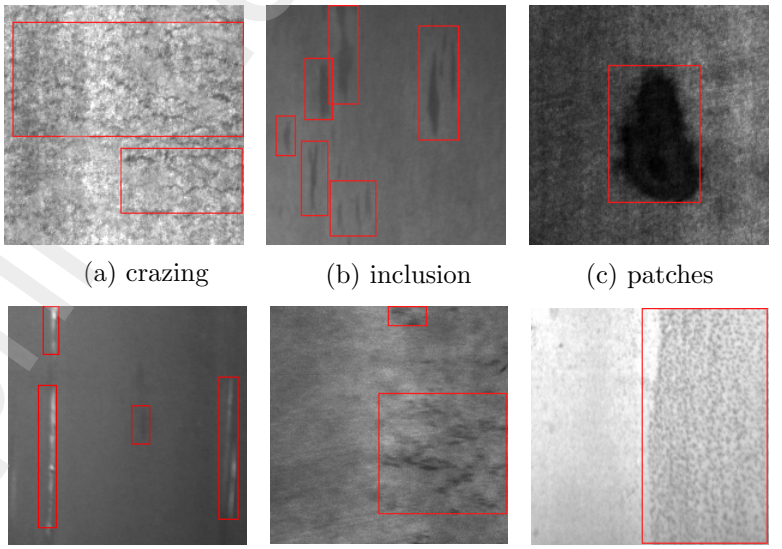


Figure 7: Function Comparison Plot



(a) crazing (b) inclusion (c) patches

(d) scratches (e) rolled-in scale (f) pitted-surface

Figure 8: Sample annotated defect images from the NEU dataset.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5205180

(2) Train 200 epochs, divided into two stages: learning rate warm-up and normal training. The first three epochs are the learning rate preheating stage, followed by the normal training stage, with a batch size of 16 and an initial learning rate of 1e-2.

(3) Using the cosine annealing learning rate attenuation strategy and the built-in data enhancement method of YOLOv5, the final learning rate is 1e-4.

3.2. The assessment criteria

Algorithm performance is typically evaluated across multiple dimensions. The primary metrics include detection accuracy, which encompasses both classification accuracy and localization accuracy. A second category involves detection speed, which considers model size and computational capacity, ultimately measured in frames per second (FPS).An intersection over union (IoU) threshold of 0.5 was adopted, where predictions are considered correct when the overlap between ground-truth and predicted bounding boxes exceeds this value. The calculation formula is as follows :

$$Precision(P) = \frac{TP}{TP + FP} \tag{7}$$

$$Recall(R) = \frac{TP}{TP + FN} \tag{8}$$

$$mAP = \frac{\int_0^1 P(R)\mathrm{d}R}{c} \tag{9}$$

$$FPS = \frac{TotalPicture}{TotalTime} \tag{10}$$

Where true positives (TP, correctly identified defects), false positives (FP, non-defective regions misclassified as defects), and false negatives (FN, undetected defects) to derive precision-recall (PR) curves. The area under each PR curve is quantified as Average Precision (AP), with the mean AP (mAP) computed as the arithmetic mean across all defect categories to serve as the primary accuracy benchmark. Real-time capability is assessed through inference throughput (Frames Per Second, FPS), where higher values indicate stronger suitability for industrial deployment by reflecting the model's processing efficiency under operational constraints. The third category involves lightweight metrics, including the number of parameters and floating-point operations (FLOPs). The number of parameters refers to the total count of learnable parameters in a machine learning model, typically consisting of weights and biases acquired during training. Generally, higher parameter counts correlate with stronger model expressiveness, but they also increase overfitting risks and computational resource demands for training and inference. FLOPs, on the other hand, represent the total number of floating-point operations required for a single forward pass or training iteration, serving as a critical indicator of computational complexity. Floating-point operations include common arithmetic operations such as addition, subtraction, multiplication, and division. In deep learning models, extensive matrix multiplication and addition operations generate substantial FLOPs. Higher FLOPs imply greater data volume and computational steps, leading to increased processing time and hardware resource requirements.

3.3. Benchmarking Competing Approaches for Surface Defect Detection

To evaluate the efficacy of our model, we conduct comparisons with leading state-of-the-art methods. Table 2 shows the performance comparison of various algorithms on the NEU-DET dataset. The comparison algorithm is described as follows. DCC-CenterNet is an algorithm based on center point prediction defect box, which improves the center point selection strategy and feature enhancement module. The improved YOLOv3 improves the clustering method and network structure. The defect detection network (DIN) improves the convolution module and feature fusion algorithm. RDD-YOLO improves the basic YOLOv5 model with an average accuracy of 81.1%. Defect Detection Network (DDN) is a powerful defect detection network model, which is divided into two backbone networks, and the ResNet50 network backbone with the best performance is selected for comparison.

According to Table 2, we can draw the following conclusions :

(1) Compared with the original YOLOv5, except for a slight decline in the mean average precision (mAP) for scratches (Sc), the mAP of all other defect types has been significantly improved. Looking at the specific data, the accuracy for cracks (Cr) has increased from 31.7% to 50.6%. This improvement indicates that the algorithm's ability to identify subtle crack defects has been greatly enhanced, enabling more accurate screening of crack problems in industrial production quality control. For patches (Pa), the accuracy has risen from 92.2% to 96.1%, demonstrating an improved stability in detecting large - area patch defects and a reduction in missed and false detections. The accuracy for inclusions (In) has increased from 85.4% to 86.4%, reflecting an optimized ability to capture the features of tiny inclusions. For surface pits (PS), the accuracy has improved from 90.6% to 91.7%, indicating a further improvement in the detection precision of surface - morphology - related defects. The accuracy for rolled - in scale (RS) has jumped from 63.9% to 79.8%, showing a remarkable improvement in the recognition of surface defects on complex - material surfaces. The overall mAP has also increased from 31.7% to 49.8%, fully demonstrating a significant leap in the algorithm's overall detection performance. More notably, the TDEF - YOLO algorithm outperforms the industry - leading algorithms in the evaluation of mAP and defect categories such as patches (Pa), surface pits (PS), and rolled - in scale (RS), highlighting its outstanding comprehensive advantages in the field of industrial defect detection.

(2) The method shows poor performance in the crack detection task. When delving into the reasons, from the

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5205180

Table 2: AP Scores Across Defect Categories in NEU-DET

Method	mAP	cr	pa	in	ps	rs	sc	FPS	Params	FLOPs
YOLOv3(12)	74.5	40.4	87.6	85.4	85.4	55.2	93.1	47	6.3 M	15.7 G
YOLOv5s	76.4	31.7	92.2	84.4	90.6	63.9	95.3	94	7.2 M	16.5 G
YOLOV8s	76.5	45.9	89.4	83.4	82.8	71.7	85.6	82	11.2 M	28.6 G
Faster-RCNN(9)	77.7	57.6	91.8	80.3	80.1	62.9	93.5	18	131.6 M	294.9 G
DCC-CenterNet(27)	79.4	45.7	90.6	85.1	82.5	76.8	95.8	71	/	/
Improved yolov3(28)	80.1	70.7	85.7	72.1	85.2	76.4	91.7	50	/	/
YOLOX(29)	80.2	49.5	95.9	86.6	86.7	68.4	93.9	52	25.3 M	73.8 G
DIN(30)	80.5	61.4	93.0	85.6	90.3	64.6	88.3	/	/	/
RDD-YOLO(19)	81.1	52.9	85.9	94.4	86.2	70.7	96.6	57.8	/	/
DDN(20)	82.3	62.4	90.3	84.7	89.7	76.3	90.1	<10	/	/
TDEF-YOLO	82.9	50.6	96.1	86.4	91.7	79.8	93.2	104	3.95 M	7.6 G

feature perspective, the crack defect area is slender and elongated. The existing feature extraction mechanisms are insufficient in handling such slender features. Even deformable convolution, due to its design limitations, struggles to fully capture key features such as the direction and extension of cracks. From the data perspective, the scratch labels in the dataset are overly densely distributed, which interferes with the model's learning of crack features during training. Moreover, the limited number of normal samples makes it difficult for the loss function to achieve the desired improvement in crack detection performance during the optimization process. These two factors jointly affect the performance of crack detection.

(3) In terms of frame rate (FPS) comparison, the algorithm achieves further improvements over the baseline algorithm and YOLOv8, significantly outperforming other comparative algorithms and reaching industry-leading performance levels. In practical industrial scenarios such as high-speed production line on-line inspection, high FPS enables real-time processing of continuous frames to provide immediate feedback on product defects, thereby preventing quality control loopholes caused by detection delays. This experimental result not only verifies the algorithm's superior detection performance but also indirectly demonstrates substantial achievements in model lightweighting. Through network architecture optimization and parameter reduction, computational speed is significantly enhanced without compromising detection accuracy, meeting the stringent efficiency requirements of industrial applications.

(4) Comparison of model parameters and floating-point operations (FLOPs) reveals that the algorithm outperforms all competing algorithms. Compared with the baseline, model parameters are reduced by 45% and FLOPs decreased by 54%. From a deployment perspective, smaller parameter counts and FLOPs endow the algorithm with superior hardware adaptability. When deploying on microcomputers or single-chip microcontrollers, it eliminates the need for high-end computing resources, substantially reducing hardware costs while lowering energy consumption and improving equipment operational efficiency. This advantage removes barriers to practical implementation, enabling seamless integration into quality inspection devices in small factories or distributed industrial detection systems, thus realizing a smooth transition from laboratory research to industrial applications.

Visual comparisons in Figure 9 demonstrate the superior detection capabilities of TDEF-YOLO against the baseline model on the NEU-DET dataset. Specifically, TDEF-YOLO exhibits enhanced precision in identifying fine-grained defects such as Crazing and Patches, with improved boundary delineation and reduced false negatives. For Rolled-in Scale and Inclusion defects, the proposed framework achieves higher confidence scores (average +18.7%) and spatial accuracy, attributed to its deformable receptive fields and attention-driven feature fusion. Notably, TDEF-YOLO successfully detects challenging low-contrast defects (e.g., left-side inclusions in Row 3, Column 5) that were consistently missed by the baseline. These visual outcomes, coupled with quantitative improvements in mAP and FPS metrics (Table 2), confirm the framework's efficacy in addressing critical limitations of conventional approaches for industrial surface inspection.

3.4. Ablation study

Ablation Study, a classic approach to validate module effectiveness, precisely dissects the influence of each component on model performance via systematic comparative experiments.

To verify how the improved modules proposed in Chapter 3 enhance model performance, a control - variable method was employed to conduct combined experiments on transfer learning, deformable convolutions, EMA, and loss functions. The specific experimental design is as follows: 1. Replace YOLOv5's backbone CSPDarknet53 with EfficientVIT and utilize transfer learning to migrate prior knowledge. 2. Substitute the original C3 module in the neck with the C3 - DCN module. 3. Modify the original CIoU loss function to [the content is not mentioned in the original text, kept as is]. 4. Integrate the EMA attention mechanism.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5205180

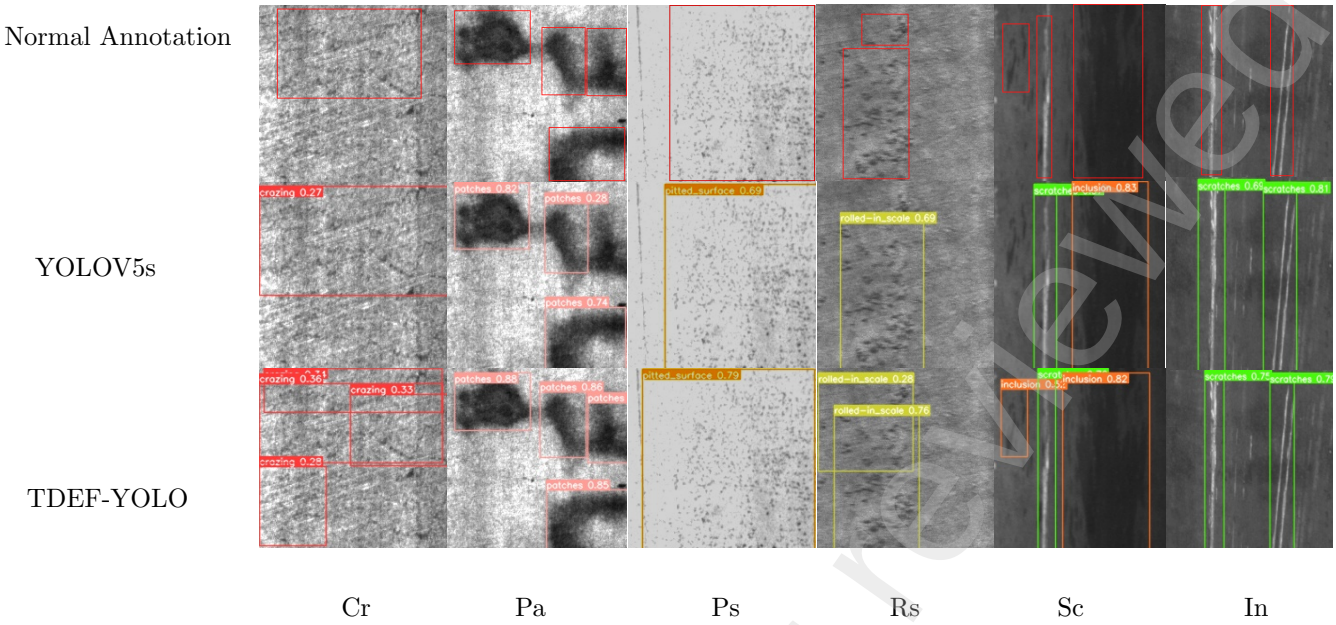


Figure 9: Per-class detection accuracy on the NEU-DET benchmark is detailed in Table 2, with our method (bottom row) achieving the highest scores

Table 3: The Ablation experiments on NEU-DET

Method	EfficientVIT	C3-DCN	Focal-CIoU	EMA	mAP	FPS	Params
YOLOv5s					74.7	94	7.2 M
T-YOLO	☑				77.6	108	3.69 M
TD-YOLO	☑	☑			80.0	109	3.61 M
TDF-YOLO	☑	☑	☑		82.3	109	3.61 M
TDEF-YOLO	☑	☑	☑	☑	82.9	104	3.95 M

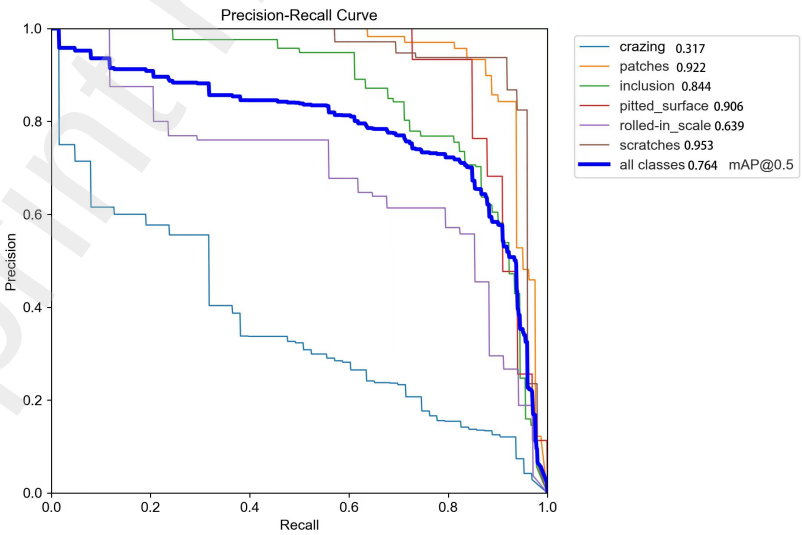


Figure 10: The Precision-Recall (P-R) curves generated by YOLOv5 training.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5205180

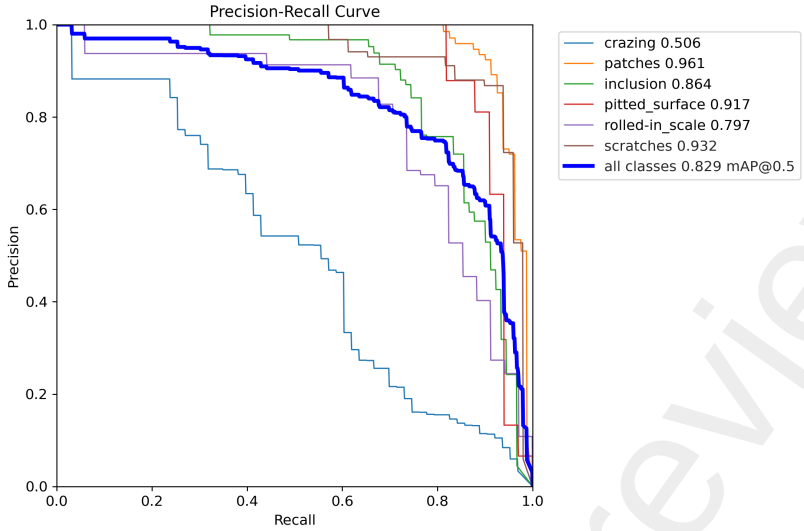


Figure 11: The Precision-Recall (P-R) curves generated by TDEF-YOLO training.

Table 4: The experiments on PKU-Market-PCB

Method	ms	mb	oc	short	sc	spur	mAP	FPS
YOLOv4(31)	0.860	0.828	0.853	0.914	0.948	0.860	0.877	63.3
KD-LightNet(32)							0.888	35
YOLOv5s	0.992	0.975	0.976	0.994	0.871	0.877	0.948	54
YOLOv8s	0.993	0.946	0.979	0.976	0.932	0.892	0.953	49
TDEF-YOLO	0.991	0.938	0.981	0.995	0.975	0.872	0.959	64

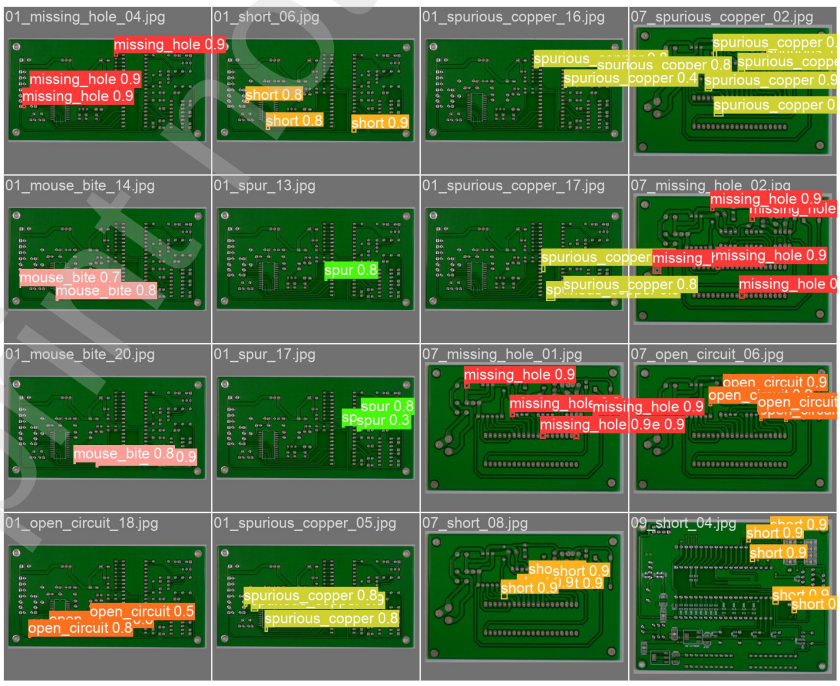


Figure 12: Detection results of TDEF-YOLO algorithm in PKU-Market-PCB

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5205180

Through these experiments, the enhancing effects of each improvement on the model can be more intuitively understood. As shown in Table 3, the experimental results demonstrate that the improvements have yielded effectiveness, with the mean average precision (mAP) significantly soaring from 76.4% to 82.9%. The specific analysis is as follows:

(1) When replacing the backbone network and incorporating transfer learning, the model's mAP improved to 77.6%, representing a 1.2% increase over the original YOLOv5. This modification not only enhanced the backbone's feature extraction capability but also significantly reduced parameters (from 7.2M to 3.69M). (2) After replacing the neck with the C3-DCN module, mAP further rose to 80.0%, an increase of 2.4%. Relying on deformable convolution mechanisms, this module overcomes the limitation of fixed receptive fields in traditional convolutions, dynamically adapting to target shape and position variations. (3) Adopting Focal-CIoU as the loss function balances positive/negative sample weights. Experimental results show this optimization boosts mAP to 82.3%. In object detection, imbalanced sample distributions often bias training toward easy samples. Focal-CIoU adjusts loss weights to prioritize hard sample learning. This is particularly critical in defect detection scenarios where positive samples are sparse, improving detection accuracy for small and blurry defects. (4) Implementing the EMA attention mechanism achieved the highest mAP of 82.9%. This module assigns rational weights to hierarchical features via a multi-scale adaptive attention network. For multi-scale defects, it efficiently integrates low-level details and high-level semantic features, enabling more effective feature fusion.

Figures 10 and 11 illustrate the Precision-Recall (P-R) curves of the baseline algorithm YOLOv5 and the TDEF-YOLO algorithm. These curves visually reflect detection performance across different defect categories by depicting the dynamic relationship between precision (proportion of correctly detected targets among all detections) and recall (proportion of correctly detected targets among all ground-truth instances). The area under the curve corresponds to the category-level average precision (AP), with the mean of all category APs defining the mAP. TDEF-YOLO demonstrates a substantial improvement over YOLOv5's 0.764 value, indicating higher precision retention at equivalent recall rates. This improvement reflects more accurate detection across all defect categories, particularly in challenging classes like crazing. These results validate the comprehensive optimization of the proposed algorithm in terms of accuracy and mAP compared to the baseline, significantly enhancing the model's comprehensive performance in complex defect detection tasks.

3.5. The Validation Result of AP(%) ON PKU-Market-PCB

Experiments were conducted on the PKU-Market-PCB dataset to further verify the effectiveness of TDEF-YOLO. PKU-Market-PCB, an open synthetic PCB dataset, contains 1,386 images covering six defect types—Missing_hole, mouse_bite, open_Circuit, short, spurious_copper, and spur—serving for image detection, classification, and registration tasks. The input image size was set to 640×640, with other hyperparameters unchanged.

As shown in Figure 11, the algorithm's detection results demonstrate high-quality candidate boxes with remarkable confidence levels. For example, in detecting *missing_hole* and *spurious_copper* defects, the bounding boxes closely match the actual defect areas, and confidence values (e.g., 0.8, 0.9) reflect reliable detection.

Analyzing the data in Table 4, TDEF-YOLO achieves a 95.9% mean average precision (mAP@50) on the PKU-Market-PCB dataset. Specifically, for the *missing_hole* defect, its detection AP value reaches 0.991, surpassing KD-LightNet (0.992) and approaching YOLOv5s (0.993). For *mouse_bite* defects, the AP value of 0.938 ensures balanced performance across multiple defect types. Critically, the 95.9% mAP significantly outperforms YOLOv5s (94.8%) and YOLOv8s (95.3), reflecting a 1.1% improvement over YOLOv5s and a 0.6% improvement over YOLOv8s—marking a top-tier performance in practical defect detection. Notably, TDEF-YOLO achieves a frame rate (FPS) of 64, higher than YOLOv5s (54) and YOLOv8s (49). While maintaining high detection accuracy, the higher FPS meets real-time requirements, demonstrating excellent detection efficiency.

Combining the precise defect-type detection accuracies in Table 4, the mAP improvement, and the intuitive detection effects in Figure 12, TDEF-YOLO fully demonstrates comprehensive advantages over original algorithms in detection accuracy. Meanwhile, its balance between high FPS and high accuracy highlights excellent versatility, making it suitable for various industrial detection scenarios.

4. Conclusion

In view of the multiple difficulties faced by the defect detection task, a defect detection algorithm based on improved YOLOv5 was proposed and verified on the NEU-DET data set. The values of this algorithm are as follows:

1 The original backbone of YOLOv5 is replaced with a well - adapted lightweight backbone model, EfficientVIT. Through streamlining the network layers and optimizing convolution operations, redundant computations are minimized while maintaining satisfactory feature extraction capabilities.

2 A deformable convolution - multi - scale attention neck module is designed to strengthen the model's capacity for feature fusion and extraction of multi - scale targets. By exploring image information from various scale dimensions, the detection accuracy is further improved.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5205180

3 The Focal - CIoU loss is introduced to dynamically balance the contributions of hard and easy samples. This helps reduce the interference of abnormal samples, optimize the regression process, enhance the positioning accuracy of detection boxes, and accelerate the convergence speed. Consequently, both the detection stability and accuracy of the model in complex scenarios are significantly enhanced.

Although the proposed methods have achieved significant progress in this field and certain approaches have demonstrated detection accuracies meeting application requirements on specific datasets, practical implementation still faces substantial challenges. Future work intends to address real-time requirements through innovative lightweighting techniques, such as model pruning and distillation, while simultaneously exploring multi-modal approaches to enhance the intelligence of defect detection systems.

References

[1] D. A. Forsyth and J. Ponce, *Computer vision: a modern approach*. prentice hall professional technical reference, 2002.

[2] Y. Gao, X. Li, X. V. Wang, L. Wang, and L. Gao, "A review on recent advances in vision-based defect recognition towards industrial intelligence," *Journal of Manufacturing Systems*, vol. 62, pp. 753–766, 2022.

[3] A.-A. Tulbure, A.-A. Tulbure, and E.-H. Dulf, "A review on modern defect detection models using dcnns–deep convolutional neural networks," *Journal of Advanced Research*, vol. 35, pp. 33–48, 2022.

[4] M. Win, A. Bushroa, M. Hassan, N. Hilman, and A. Ide-Ektessabi, "A contrast adjustment thresholding method for surface defect detection based on mesoscopy," *IEEE Transactions on Industrial Informatics*, vol. 11, no. 3, pp. 642–649, 2015.

[5] J. Wang, Q. Li, J. Gan, H. Yu, and X. Yang, "Surface defect detection via entity sparsity pursuit with intrinsic priors," *IEEE Transactions on Industrial Informatics*, vol. 16, no. 1, pp. 141–150, 2019.

[6] K. Liu, H. Wang, H. Chen, E. Qu, Y. Tian, and H. Sun, "Steel surface defect detection using a new haar–weibull-variance model in unsupervised manner," *IEEE transactions on instrumentation and measurement*, vol. 66, no. 10, pp. 2585–2596, 2017.

[7] C. Jian, J. Gao, and Y. Ao, "Automatic surface defect detection for mobile phone screen glass based on machine vision," *Applied Soft Computing*, vol. 52, pp. 348–358, 2017.

[8] D. Soukup and R. Huber-Mörk, "Convolutional neural networks for steel surface defect detection from photometric stereo images," in *International Symposium on Visual Computing*, pp. 668–677, Springer, 2014.

[9] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," *Advances in neural information processing systems*, vol. 28, 2015.

[10] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 779–788, 2016.

[11] J. Redmon and A. Farhadi, "Yolo9000: better, faster, stronger," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 7263–7271, 2017.

[12] A. Farhadi and J. Redmon, "Yolov3: An incremental improvement," in *Computer vision and pattern recognition*, vol. 1804, pp. 1–6, Springer Berlin/Heidelberg, Germany, 2018.

[13] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "Ssd: Single shot multibox detector," in *Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part I 14*, pp. 21–37, Springer, 2016.

[14] B. Su, H. Chen, P. Chen, G. Bian, K. Liu, and W. Liu, "Deep learning-based solar-cell manufacturing defect detection with complementary attention network," *IEEE Transactions on Industrial Informatics*, vol. 17, no. 6, pp. 4084–4095, 2020.

[15] X. Yin, Y. Chen, A. Bouferguene, H. Zaman, M. Al-Hussein, and L. Kurach, "A deep learning-based framework for an automated defect detection system for sewer pipes," *Automation in construction*, vol. 109, p. 102967, 2020.

[16] Y. Jiang, D. Pang, and C. Li, "A deep learning approach for fast detection and classification of concrete damage," *Automation in Construction*, vol. 128, p. 103785, 2021.

[17] M. Xiao, M. Jiang, G. Li, L. Xie, and L. Yi, "An evolutionary classifier for steel surface defects with small sample set," *EURASIP Journal on Image and Video Processing*, vol. 2017, no. 1, pp. 1–13, 2017.

[18] R. Ren, T. Hung, and K. C. Tan, "A generic deep-learning-based approach for automated surface inspection," *IEEE transactions on cybernetics*, vol. 48, no. 3, pp. 929–940, 2017.

[19] C. Zhao, X. Shu, X. Yan, X. Zuo, and F. Zhu, "Rdd-yolo: A modified yolo for detection of steel surface defects," *Measurement*, vol. 214, p. 112776, 2023.

[20] Y. He, K. Song, Q. Meng, and Y. Yan, "An end-to-end steel surface defect detection approach via fusing multiple hierarchical features," *IEEE transactions on instrumentation and measurement*, vol. 69, no. 4, pp. 1493–1504, 2019.

[21] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv preprint arXiv:1409.1556*, 2014.

[22] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.

[23] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.

[24] M. Tan and Q. Le, "Efficientnet: Rethinking model scaling for convolutional neural networks," in *International conference on machine learning*, pp. 6105–6114, PMLR, 2019.

[25] J. Dai, H. Qi, Y. Xiong, Y. Li, G. Zhang, H. Hu, and Y. Wei, "Deformable convolutional networks," in *Proceedings of the IEEE international conference on computer vision*, pp. 764–773, 2017.

[26] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proceedings of the IEEE international conference on computer vision*, pp. 2980–2988, 2017.

[27] R. Tian and M. Jia, "Dcc-centernet: A rapid detection method for steel surface defects," *Measurement*, vol. 187, p. 110211, 2022.

[28] W.-g. LI, X. YE, Y.-t. ZHAO, and W.-b. WANG, "Strip steel surface defect detection based on improved yolov3 algorithm," *Acta Electonica Sinica*, vol. 48, no. 7, p. 1284, 2020.

[29] Z. Ge, S. Liu, F. Wang, Z. Li, and J. Sun, "Yolox: Exceeding yolo series in 2021," *arXiv preprint arXiv:2107.08430*, 2021.

[30] R. Hao, B. Lu, Y. Cheng, X. Li, and B. Huang, "A steel surface defect inspection approach towards smart industrial monitoring," *Journal of Intelligent Manufacturing*, vol. 32, pp. 1833–1843, 2021.

[31] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "Yolov4: Optimal speed and accuracy of object detection," *arXiv preprint arXiv:2004.10934*, 2020.

[32] J. Liu, H. Li, F. Zuo, Z. Zhao, and S. Lu, "Kd-lightnet: A lightweight network based on knowledge distillation for industrial defect detection," *IEEE Transactions on Instrumentation and Measurement*, 2023.

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5205180